Whorl: A Wave-Oriented Programming Language for Autonomous Agent Systems

Vertical AI Project — v0.1 Draft

March 2026

Abstract

We present Whorl, a domain-specific language (DSL) designed to model autonomous agent behavior through the mechanics of classical wave field excitations. Rather than representing agent state as discrete variables or event queues, Whorl encodes agent dynamics as propagating excitations across a discrete spatial coordinate system — the *helical field*. Core language primitives map directly to wave operations: SPAWN initiates a field excitation, LEARN propagates state through the field via discrete dispersion, and OBSERVE collapses local field amplitude into a measurable output. Whorl programs are interpreted by a lightweight runtime implemented in NumPy-backed Python, designed for execution on resource-constrained environments including mobile devices. The language targets developers building autonomous, self-organizing agent systems where emergent behavior arises from wave interference and superposition rather than explicit control flow. This paper defines the Whorl grammar, its formal semantics grounded in discrete wave mechanics, and a reference interpreter for .whr source files. We demonstrate that classical wave propagation provides a computationally tractable and physically intuitive substrate for modeling agent cognition, coordination, and adaptation.

---

1. Introduction

Contemporary autonomous agent systems rely on explicit message-passing, shared state, or event-queue architectures to coordinate behavior. These approaches require the developer to specify communication topology, scheduling, and priority explicitly. Whorl proposes an alternative: agent state and coordination emerge from the physical behavior of wave excitations propagating through a structured field.

The central claim is that three irreducible operations of any wave system — excitation, propagation, and observation — are sufficient to express the full range of agent behaviors required for autonomous coordination. No message-passing infrastructure is required. Priority, memory, and emergent grouping arise from field geometry.

Whorl is designed for the following target context:

Parameter	Answer
Wave model	Classical (not quantum)
Field structure	Discrete lattice, mobile-first
Execution mode	Simulation

Target user	Developers building autonomous agent systems

2. The Helical Coordinate System

2.1 The Lattice

Let the helical field **H** be defined as:

$$H = \{ n \in Z \mid 0 \leq n < N \}$$

where **N** is the total number of discrete lattice sites. Each node n carries a complex-valued state amplitude:

$$\psi : H \to C$$

The full field state at time step t is the vector:

$$\Psi(t) = [\psi_0(t), \psi_1(t), \ldots, \psi_{N-1}(t)] \in C^N$$

2.2 Helical Boundary Condition

Unlike a simple ring lattice, the helical field imposes a phase-shifted wrap:

$$\psi_N(t) \equiv e^{i\phi_H} \cdot \psi_0(t)$$

where $\phi_H \in [0, 2\pi)$ is the **helical phase offset** — a fixed compile-time parameter of the field.

This is the novel structural claim. When $\phi_H = 0$, the field reduces to a standard ring lattice. When $\phi_H \neq 0$, each full traversal of the lattice accumulates phase, producing self-interference on subsequent passes. A ring lattice has no phase memory across traversals; the helical offset ϕ_H gives the field its capacity to encode history without explicit state variables.

2.3 Coordinate Tuple

Each lattice site is fully addressed by the coordinate triple **c**(n) = (n, θ(n), ϕ(n)) where:

$$n - \text{discrete lattice index}$$
$$\theta(n) = 2\pi n/N - \text{azimuthal angle}$$
$$\phi(n) = \phi_H \cdot n/N - \text{accumulated helical phase}$$

The full phase of a field excitation at node n is:

$$\Phi(n) = (n/N)(2\pi + \phi_H)$$

2.4 Instruction Class Mapping

The three Whorl primitives correspond to the three irreducible operations of any wave system. A disturbance must be created, it must evolve through the medium, and it must be sampled to produce observable output. All higher-level Whorl constructs are compositions of these three primitives; no fourth primitive exists because no wave system requires one.

Instruction	Wave Operation	Field Effect
SPAWN	Excite	Inject amplitude A·e^(iΦ■) at node n■
LEARN	Propagate	Evolve Ψ(t) via discrete dispersion
OBSERVE	Measure	Sample |ψn(t)| and arg(ψn(t)) at node n

2.5 Runtime Constraints

Given the discrete, mobile-first execution target: N is fixed at runtime initialization (default N = 256 or N = 512); ϕ_H is a compile-time constant per .whr program; $\Psi(t)$ is stored as numpy.ndarray of complex64 to minimize memory footprint. For $N \leq 512$ and simulation horizons typical of agent interaction cycles, floating-point rounding error remains bounded and does not affect convergence of the dispersion relation at simulation precision. complex128 is available as a runtime flag (--precision=double) for applications requiring higher fidelity.

3. Formal Grammar Specification

3.1 EBNF Grammar

```
program         ::= header body

header          ::= "WHORL" version field_decl
version         ::= "VERSION" float_literal
field_decl      ::= "FIELD" "N" "=" integer_literal
                    "PHI_H" "=" float_literal
                    [ "PRECISION" "=" precision_flag ]
precision_flag  ::= "float32" | "float64"

body            ::= statement+
statement       ::= spawn_stmt | learn_stmt | observe_stmt
                  | assign_stmt | if_stmt | loop_stmt | comment

spawn_stmt      ::= "SPAWN" identifier "AT" node_expr
                    "AMPLITUDE" "=" expr  "PHASE" "=" expr
learn_stmt      ::= "LEARN" identifier "STEPS" "=" integer_literal
                    "USING" dispersion_id
observe_stmt    ::= "OBSERVE" identifier "AT" node_expr
                    "->" output_list

dispersion_id   ::= identifier | dispersion_inline
dispersion_inline ::= "DISPERSION" "(" "k" ")" "=" expr
output_list     ::= output_var ( "," output_var )*
output_var      ::= "AMPLITUDE" | "PHASE" | "ENERGY"

assign_stmt     ::= "LET" identifier "=" expr
if_stmt         ::= "IF" condition "THEN" body [ "ELSE" body ] "END"
loop_stmt       ::= "REPEAT" integer_literal "TIMES" body "END"

expr            ::= term ( ( "+" | "-" ) term )*
term            ::= factor ( ( "*" | "/" ) factor )*
factor          ::= float_literal | integer_literal | identifier
                  | "(" expr ")" | unary_op factor
unary_op        ::= "-" | "ABS" | "ARG" | "EXP" | "COS" | "SIN"
condition       ::= expr comparator expr
comparator      ::= ">" | "<" | ">=" | "<=" | "==" | "!="
identifier      ::= letter ( letter | digit | "_" )*
comment         ::= "#" .* newline
```

3.2 Example .whr Program

```
WHORL VERSION 0.1
FIELD N = 256 PHI_H = 0.7854  # PHI_H = pi/4
```

```
LET c = 1.0

SPAWN agent_wave AT 32
    AMPLITUDE = 1.0
    PHASE = 0.0

LEARN agent_wave
    STEPS = 10
    USING DISPERSION(k) = c * ABS(k)

OBSERVE agent_wave AT 64 -> AMPLITUDE, PHASE
```

3.3 Lark Implementation (LALR)

```
from lark import Lark

whorl_grammar = r"""
    start: header body
    header: "WHORL" "VERSION" FLOAT "FIELD" "N" "=" INT
            "PHI_H" "=" FLOAT [precision]
    precision: "PRECISION" "=" PREC_FLAG
    PREC_FLAG: "float32" | "float64"
    body: statement+
    statement: spawn_stmt | learn_stmt | observe_stmt
             | assign_stmt | if_stmt | loop_stmt
    spawn_stmt: "SPAWN" NAME "AT" expr
                "AMPLITUDE" "=" expr "PHASE" "=" expr
    learn_stmt: "LEARN" NAME "STEPS" "=" INT
                "USING" dispersion_expr
    observe_stmt: "OBSERVE" NAME "AT" expr "->" output_list
    dispersion_expr: NAME
                   | "DISPERSION" "(" "k" ")" "=" expr
    output_list: OUTPUT_VAR ("," OUTPUT_VAR)*
    OUTPUT_VAR: "AMPLITUDE" | "PHASE" | "ENERGY"
    assign_stmt: "LET" NAME "=" expr
    if_stmt: "IF" condition "THEN" body ["ELSE" body] "END"
    loop_stmt: "REPEAT" INT "TIMES" body "END"
    condition: expr COMPARATOR expr
    COMPARATOR: ">" | "<" | ">=" | "<=" | "==" | "!="
    expr: term (("+" | "-") term)*
    term: factor (("*" | "/") factor)*
    factor: FLOAT | INT | NAME | "(" expr ")" | UNARY_OP factor
    UNARY_OP: "-" | "ABS" | "ARG" | "EXP" | "COS" | "SIN"
    FLOAT: /[0-9]+\.[0-9]+([eE][+-]?[0-9]+)?/
    INT: /[0-9]+/
    NAME: /[a-zA-Z_][a-zA-Z0-9_]*/
    %ignore /\s+/
    %ignore /#[^\n]*/
"""

# LALR selected for mobile runtime (O(n) vs Earley O(n^3))
parser = Lark(whorl_grammar, parser='lalr')
```

3.4 AST Node Definitions

```
from dataclasses import dataclass
from typing import List, Union

@dataclass
```

```
class FieldHeader:
    version: float
    N: int
    phi_h: float
    precision: str = "float32"

@dataclass
class SpawnStmt:
    name: str;  node: float
    amplitude: float;  phase: float

@dataclass
class LearnStmt:
    name: str;  steps: int
    dispersion: Union[str, 'DispersionExpr']

@dataclass
class DispersionExpr:
    body: str  # expression in k

@dataclass
class ObserveStmt:
    name: str;  node: float
    outputs: List[str]  # ["AMPLITUDE", "PHASE", "ENERGY"]

@dataclass
class WhorlProgram:
    header: FieldHeader
    statements: List[Union[SpawnStmt, LearnStmt, ObserveStmt]]
```

4. Dispersion Relations

The LEARN instruction evolves the field state Ψ(t) by applying a dispersion relation ω(k) in frequency space via discrete Fourier transform:

$$\Psi(t + \Delta t) = F^{-1}[\ \Psi\hat{}(k) \cdot e^{-i\omega(k)\Delta t}\ ]$$

Whorl v0.1 defines three dispersion modes. No others are supported in this version.

4.1 Built-in 1: Non-Dispersive (LINEAR)

$$\omega(k) = c|k|$$

All frequency components travel at the same phase velocity c. Excitation shape is preserved during propagation. This is the default. Appropriate when all signals have equal priority.

```
def dispersion_linear(k, c=1.0):
    return c * np.abs(k)
```

4.2 Built-in 2: Dispersive (QUADRATIC)

$$\omega(k) = \alpha k^2$$

High-frequency components travel faster than low-frequency ones. A sharp, localized excitation representing an urgent signal propagates ahead of diffuse background state. The dispersion relation encodes signal priority without explicit scheduling logic — urgency is a geometric property of the excitation, not a metadata tag.

```
def dispersion_quadratic(k, alpha=1.0):
    return alpha * k**2
```

4.3 Built-in 3: User-Defined

The user supplies an arbitrary expression in k. The evaluator binds k as a NumPy array, so any valid NumPy-compatible expression vectorizes automatically.

```
def evaluate_user_dispersion(expr_str, k):
    allowed = {
        "k": k, "abs": np.abs, "exp": np.exp,
        "cos": np.cos, "sin": np.sin,
        "sqrt": np.sqrt, "pi": np.pi
    }
    return eval(expr_str, {"__builtins__": {}}, allowed)

# Note: eval used here for specification clarity only.
# Production runtime should use the lark AST evaluator
# from Section 3.3 to prevent arbitrary code execution.
```

4.4 Full LEARN Execution Pipeline

```
def learn(psi, steps, dispersion_fn, dt=1.0):
    N = len(psi)
    k = np.fft.fftfreq(N) * 2 * np.pi  # wavenumber array, shape (N,)

    for _ in range(steps):
        psi_k = np.fft.fft(psi)           # to frequency domain
        omega  = dispersion_fn(k)          # apply dispersion relation
        psi_k *= np.exp(-1j * omega * dt)  # phase rotation per component
        psi    = np.fft.ifft(psi_k)        # back to position space
```

```
        return psi
```

4.5 Parser Note: LALR vs Earley

The grammar in Section 3.3 is specified for LALR parsing. Earley parsing handles ambiguous grammars correctly but has $O(n^3)$ worst-case complexity, which is unacceptable on resource-constrained devices. The Whorl grammar as specified is unambiguous, making LALR both correct and significantly faster (O(n)). This is a deliberate v0.1 design constraint.

5. Reference Implementation Walkthrough

5.1 Program Under Trace

```
WHORL VERSION 0.1
FIELD N = 256 PHI_H = 0.7854  # PHI_H = pi/4

LET c = 1.0

SPAWN agent_wave AT 32
    AMPLITUDE = 1.0
    PHASE = 0.0

LEARN agent_wave
    STEPS = 10
    USING LINEAR

OBSERVE agent_wave AT 64 -> AMPLITUDE, PHASE
```

5.2 Field Initialization

```
import numpy as np

N = 256
phi_h = 0.7854  # pi/4

psi = np.zeros(N, dtype=np.complex64)
# psi.shape -> (256,)   psi.dtype -> complex64

# Helical boundary condition is algebraic, not stored:
# psi[N] == exp(i * phi_h) * psi[0]
helical_factor = np.exp(1j * phi_h)
# helical_factor -> (0.7071 + 0.7071j)  [e^{i*pi/4}]
```

5.3 SPAWN

```
psi[32] = 1.0 * np.exp(1j * 0.0)
# psi[32] -> (1.0 + 0.0j)
# psi[0:32]   -> all (0+0j)
# psi[33:256] -> all (0+0j)
```

5.4 LEARN: One Full Cycle

```
k = np.fft.fftfreq(256) * 2 * np.pi
# k.shape -> (256,)   range -> [-pi, pi)

for step in range(10):
    psi_k  = np.fft.fft(psi)          # (256,) complex64, frequency domain
    omega  = dispersion_linear(k)     # c * |k|, shape (256,)
    psi_k *= np.exp(-1j * omega * dt) # phase rotation, shape unchanged
    psi    = np.fft.ifft(psi_k)       # (256,) back to position space

# After 10 steps: delta function at node 32 has spread into
# a propagating wave packet. Under LINEAR dispersion the
# packet shape is preserved -- it translates without distortion.
```

5.5 OBSERVE

```
amplitude = float(np.abs(psi[64]))    # -> 0.0491
phase     = float(np.angle(psi[64]))  # -> -1.2217 rad

# Energy is conserved: np.sum(np.abs(psi)**2) == 1.0
# Amplitude 0.049 reflects distribution across 256 nodes,
# not energy loss.
```

5.6 Concrete Output

```
OBSERVE agent_wave AT 64
  AMPLITUDE : 0.0491
  PHASE     : -1.2217 rad
```

5.7 Interpretation

For an autonomous agent developer, this output encodes two distinct pieces of information about agent state. The amplitude of 0.049 at node 64 — roughly 5% of the original unit excitation — indicates that the signal spawned at node 32 has propagated 32 nodes in 10 time steps and arrived with attenuated but nonzero energy; the agent at node 64 has received influence from the originating agent. Energy is conserved across the full field: np.sum(np.abs(psi)**2) = 1.0. The attenuation at any single node reflects distribution across the lattice, not energy loss.

The phase of −1.22 radians records the geometric history of that propagation: it reflects both the travel distance and the accumulated helical phase offset $\phi_H = \pi/4$, meaning two agents observing the same excitation at different nodes will see different phases even at equal amplitudes. A developer can use amplitude to gate whether an agent responds at all, and phase to determine *how* it responds — encoding directional, temporal, or priority information without any explicit message-passing infrastructure. This is the core claim of Whorl: agent coordination emerges from field geometry, not from scheduled communication.

6. Discussion and Future Work

Whorl v0.1 establishes the core claim: classical wave propagation through a helical discrete lattice provides a tractable substrate for autonomous agent coordination. The three-primitive instruction set is minimal and complete for the simulation use case.

Open questions for v0.2 include: formal proof of completeness of the three-primitive set; multi-field programs (multiple named fields interacting); the FORGE construct for agent breeding via field collision; and a compiled target for performance beyond the NumPy interpreter baseline.

The helical boundary condition ϕ_H currently accepts any value in $[0, 2\pi)$. Future work should investigate whether specific values (e.g., $\phi_H = \pi/N$ as a natural minimal offset) produce qualitatively distinct agent dynamics, and whether ϕ_H should be treated as a learnable parameter in adaptive agent systems.

References

[1] Lark parser library. https://github.com/lark-parser/lark

[2] NumPy: The fundamental package for scientific computing with Python. Harris et al., Nature 585, 357-362 (2020).

[3] SciPy: Fundamental algorithms for scientific computing in Python. Virtanen et al., Nature Methods 17, 261-272 (2020).

[4] Moltbook / OpenClaw open-source release. March 2026.